

✓ 1. Judul Praktikum

Praktikum Struktur Data - Modul 5: STACK (LIFO - Last In First Out)

2. Tujuan Praktikum

- Memahami cara kerja struktur data Stack beserta prinsip dasarnya.
 - Mampu menuliskan kode di Python untuk mengeksekusi operasi dasar seperti penambahan (Push) dan penghapusan (Pop).
 - Mampu mengaplikasikan konsep tumpukan ini pada pemecahan masalah logika nyata di dunia komputasi.
-

3. Dasar Teori

Stack bisa diibaratkan seperti sebuah tumpukan piring atau buku. Elemen yang paling baru ditaruh (paling atas) otomatis akan menjadi elemen pertama yang bisa diambil. Konsep antrian vertikal ini dikenal dengan istilah Last In First Out (LIFO).

Pada bahasa pemrograman Python, kita bisa memanfaatkan tipe data List untuk membuat Stack. Operasi utamanya meliputi:

- **Push:** Memasukkan data baru ke puncak tumpukan (di Python menggunakan `append()`).
- **Pop:** Menarik keluar data dari puncak tumpukan (di Python menggunakan `pop()`).
- **Peek:** Hanya mengintip data teratas tanpa mengeluarkannya.

Karena kita hanya berfokus pada satu akses pintu masuk dan keluar (ujung atas/TOP), komputasi algoritma ini sangat ringan dan cepat.

4. Langkah Praktikum

Berikut adalah implementasi dari rangkaian prosedur praktikum serta penyelesaian untuk beberapa studi kasus simulasi dunia nyata menggunakan Stack.

```
1 print("===== PROSEDUR PRAKTIKUM =====")
2
3 print("\n--- Praktikum 1: Membuat Stack ---")
4 tumpukan_dasar = []
5 tumpukan_dasar.append(10)
6 tumpukan_dasar.append(20)
7 tumpukan_dasar.append(30)
8 print("Isi Tumpukan:", tumpukan_dasar)
```

```
9
10 print("\n--- Praktikum 2: Operasi Pop ---")
11 angka_keluar = tumpukan_dasar.pop()
12 print("Angka yang ditarik =", angka_keluar)
13 print("Sisa Tumpukan =", tumpukan_dasar)
14
15 print("\n--- Praktikum 3: Operasi Peek ---")
16 print("Angka paling atas (Peek) =", tumpukan_dasar[-1])
17
18 print("\n--- Praktikum 4 & SK 2: Simulasi History Browser ---")
19 riwayat_web = []
20 riwayat_web.append("google.com")
21 riwayat_web.append("github.com")
22 riwayat_web.append("stackoverflow.com")
23 print(f"Jejak selancar web: {riwayat_web}")
24 halaman_ditutup = riwayat_web.pop()
25 print(f"User menekan 'Back' dari web: {halaman_ditutup}")
26 print(f"Posisi web saat ini: {riwayat_web[-1]}")
27
28 print("\n--- Praktikum 5 & SK 4: Pengecekan Kurung ---")
29 rumus = "((a+b)*c)"
30 cek_kurung = []
31 status_valid = True
32 for karakter in rumus:
33     if karakter == '(':
34         cek_kurung.append(karakter)
35     elif karakter == ')':
36         if len(cek_kurung) == 0:
37             status_valid = False
38             break
39         cek_kurung.pop()
40
41 if status_valid and len(cek_kurung) == 0:
42     print(f"Format rumus {rumus} -> Sesuai")
43 else:
44     print(f"Format rumus {rumus} -> Keliru")
45
46
47 print("\n===== STUDI KASUS TAMBAHAN =====")
48
49 print("\n--- SK 1: Tumpukan Buku ---")
50 koleksi_buku = []
51 koleksi_buku.append("Struktur Data")
52 koleksi_buku.append("Algoritma")
53 print("Rak saat ini:", koleksi_buku)
54 print("Buku yang sedang dibaca:", koleksi_buku.pop())
55
56 print("\n--- SK 3: Fitur Undo pada Teks ---")
57 dokumen = ""
58 jejak_ketikan = []
59
60 jejak_ketikan.append(dokumen)
61 dokumen += "Halo "
62 jejak_ketikan.append(dokumen)
63 dokumen += "Semuanya!"
64
65 print(f"Teks terketik: '{dokumen}'")
66 dokumen = jejak_ketikan.pop()
67 print(f"Teks setelah tombol Undo ditekan: '{dokumen}'")
68
69 print("\n--- SK 5: Riwayat Transaksi ATM ---")
```

```

70 mutasi_atm = []
71 mutasi_atm.append("Tarik Tunai Rp50.000")
72 mutasi_atm.append("Transfer Rp100.000")
73 mutasi_atm.append("Top Up E-Wallet Rp250.000")
74 print("Daftar Mutasi Terakhir:")
75 while len(mutasi_atm) > 0:
76     print(">>", mutasi_atm.pop())

```

===== PROSEDUR PRAKTIKUM =====

--- Praktikum 1: Membuat Stack ---
Isi Tumpukan: [10, 20, 30]

--- Praktikum 2: Operasi Pop ---
Angka yang ditarik = 30
Sisa Tumpukan = [10, 20]

--- Praktikum 3: Operasi Peek ---
Angka paling atas (Peek) = 20

--- Praktikum 4 & SK 2: Simulasi History Browser ---
Jejak selancar web: ['google.com', 'github.com', 'stackoverflow.com']
User menekan 'Back' dari web: stackoverflow.com
Posisi web saat ini: github.com

--- Praktikum 5 & SK 4: Pengecekan Kurung ---
Format rumus ((a+b)*c) -> Sesuai

===== STUDI KASUS TAMBAHAN =====

--- SK 1: Tumpukan Buku ---
Rak saat ini: ['Struktur Data', 'Algoritma']
Buku yang sedang dibaca: Algoritma

--- SK 3: Fitur Undo pada Teks ---
Teks terketik: 'Halo Semuanya!'
Teks setelah tombol Undo ditekan: 'Halo '

--- SK 5: Riwayat Transaksi ATM ---
Daftar Mutasi Terakhir:
>> Top Up E-Wallet Rp250.000
>> Transfer Rp100.000
>> Tarik Tunai Rp50.000

5. Analisis

Di Python, kita bisa merancang Stack dengan sangat mudah menggunakan metode List standar. Menjalankan perintah `append()` sama persis dengan logika operasi *Push*, karena data diletakkan di indeks yang paling ujung. Sementara itu, `pop()` akan mengeliminasi elemen paling akhir tersebut. Sangat disarankan untuk memantau sisa isi stack menggunakan fungsi `len()` agar program tidak *error* saat dipaksa menarik elemen dari stack yang sudah benar-benar kosong.

6. Kesimpulan

Sistem LIFO (Last In First Out) sangat vital untuk algoritma yang mengharuskan kita kembali ke kondisi sebelumnya (*backtracking*). Contoh nyatanya terlihat sangat jelas pada logika pembuatan tombol Undo di editor teks dan pelacakan tombol Back pada penjelajah internet.